

Scrapy commands

<code>scrapy startproject myproject [project_dir]</code>	This command will create a Scrapy project in the project directory specified; else with the name of <i>project</i> , if <i>project_dir</i> is not mentioned.
<code>scrapy genspider spider_name [domain.com]</code>	This command needs to be run from the root directory of the project, to create a spider with <i>allowed_domain</i> as <i>domain.com</i> .
<code>Scrapy bench</code>	This runs a quick benchmark test, to tell you Scrapy's maximum possible speed in crawling Web pages, given your hardware.
<code>scrapy check</code>	Checks spider contracts.
<code>scrapy crawl [spider]</code>	This command instructs the spider to start crawling the Web pages.
<code>scrapy edit [spider]</code>	This command is used to edit the spider using the editor specified in the <i>EDITOR</i> environment variables or <i>EDITOR</i> setting.
<code>scrapy fetch [url]</code>	This command downloads the contents of the URL and stores them in a standard output file.
<code>scrapy list</code>	Lists the available spiders in the project.
<code>scrapy parse [url]</code>	This is the default callback used by Scrapy to process downloaded responses, when their requests don't specify a callback.
<code>scrapy runspider file_name.py</code>	Runs a spider self-contained in a Python file without having to create a project.
<code>scrapy view [url]</code>	Opens the URL in the browser as seen by the spider.
<code>scrapy settings</code>	This is to get the Scrapy setting value.
<code>scrapy version</code>	This is to get the Scrapy version installed.
<code>scrapy shell [url optional]</code>	Opens the interactive Scrapy console for the URL.

callback to get a response.

- In the callback, after the parsing is done, either of the three dicts of content — request object, item object or iterable — is returned. This request will also contain the callback and is downloaded by Scrapy. The response is handled by the corresponding callback.
- In callbacks, parsing of page content is performed using the XPath selectors or any other parser libraries like *lxml*, and items are generated with parsed data.
- The returned items are then persisted into the database or the item pipeline, or written to a file using the *FeedExports* service.
Scrapy is bundled with three kinds of spiders.
BaseSpider: All the spiders must inherit this spider. It is the simplest one, responsible for *start_urls* / *start_request()* and calling of the parse method for each resulting response.
CrawlSpider: This provides a convenient method for crawling links by defining a set of rules. It can be overridden as per the project's needs. It supports all the *BaseSpider*'s attributes as well as an additional attribute, 'rules', which is a list of one or more rules.

XMLSpider and *CSVSpider*: *XMLSpider* iterates over the XML feeds through a certain node name, whereas *CSVSpider* is used to crawl CSV feed. The difference between them is

that *XMLSpider* iterates over nodes and *CSVSpider* iterates over rows with the *parse_rows()* method.

Having understood the different types of spiders, we are ready to start writing our first spider. Create a file named *myFirstSpider.py* in the *spiders* folder of our project.

```
import scrapy
class MyfirstspiderSpider(scrapy.Spider):
    name = "myFirstSpider"
    allowed_domains = ["opensourceforu.com"]
    start_urls = (
        'http://opensourceforu.com/2015/10/building-a-django-app/'
    )

    def parse(self, response):
        page = response.url.split("/")[-2]
        filename = 'quotes-%s.html' % page
        with open(filename, 'wb') as f:
            f.write(response.body)
            self.log('Saved file %s' % filename)
```

In the above code, the following attributes have been defined:

- name*: This is the unique name given to the spider in the project.